Coxeter groups in Lean

Ming Yean Lim

March 9, 2026

Goals

- Learn how to work with Coxeter groups in Lean

Goals

- Learn how to work with Coxeter groups in Lean
- Show examples of definitions and theorems

Goals

- Learn how to work with Coxeter groups in Lean
- Show examples of definitions and theorems
- Some potential projects

What is a Coxeter group?

Let S be a set.

What is a Coxeter group?

Let S be a set.

Definition

A *Coxeter matrix* is a matrix $m : S \times S \to \mathbb{N}_{\geq 1} \cup \{\infty\}$, such that

$$m(s, s') = m(s', s)$$
$$m(s, s') = 1 \iff s = s'$$

for all $s, s' \in S$.

What is a Coxeter group?

Let S be a set.

Definition

A *Coxeter matrix* is a matrix $m : S \times S \to \mathbb{N}_{\geq 1} \cup \{\infty\}$, such that

$$m(s, s') = m(s', s)$$
$$m(s, s') = 1 \iff s = s'$$

for all $s, s' \in S$.
m determines a *Coxeter group* W, with generators S and relations

$$(ss')^{m(s,s')} = e$$

for $s, s' \in S$ with $m(s, s') \neq \infty$.

Coxeter groups in `mathlib`

Definition

A *Coxeter matrix* is a matrix $m : S \times S \to \mathbb{N}_{\geq 1} \cup \{\infty\}$, such that

$$m(s, s') = m(s', s)$$
$$m(s, s') = 1 \iff s = s'$$

for all $s, s' \in S$.

Coxeter groups in mathlib

Definition

A *Coxeter matrix* is a matrix $m : S \times S \to \mathbb{N}_{\geq 1} \cup \{\infty\}$, such that

$$m(s, s') = m(s', s)$$
$$m(s, s') = 1 \iff s = s'$$

for all $s, s' \in S$.

Definition in mathlib:

```
structure CoxeterMatrix (B : Type*) where
  M : Matrix B B ℕ
  isSymm : M.IsSymm
  diagonal i : M i i = 1
  off_diagonal i i' : i ≠ i' → M i i' ≠ 1
```

Coxeter groups in `mathlib`

Given `M : CoxeterMatrix B`, we can get the Coxeter group with matrix `M` via `M.Group`. This is the quotient of the free group on `B` by relations.

Coxeter groups in `mathlib`

Given `M : CoxeterMatrix B`, we can get the Coxeter group with matrix `M` via `M.Group`. This is the quotient of the free group on `B` by relations.
A Coxeter group `W` with matrix `M` is a group that is isomorphic to `M.Group`.

Coxeter groups in mathlib

Given `M : CoxeterMatrix B`, we can get the Coxeter group with matrix `M` via `M.Group`. This is the quotient of the free group on `B` by relations.
A Coxeter group `W` with matrix `M` is a group that is isomorphic to `M.Group`.

```
structure CoxeterSystem {B : Type*} (M : CoxeterMatrix B) (W :
    Type*) [Group W] where
    mulEquiv : W ≃* M.Group
```

Some definitions in mathlib

```
import Mathlib.GroupTheory.Coxeter.Basic
import ...

variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

Some definitions in mathlib

```
import Mathlib.GroupTheory.Coxeter.Basic
import ...

variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

Some important functions:

- `cs.simple : B → W`

Some definitions in mathlib

```
import Mathlib.GroupTheory.Coxeter.Basic
import ...

variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

Some important functions:

- `cs.simple : B → W`
- `cs.wordProd : List B → W`

Some definitions in mathlib

```
import Mathlib.GroupTheory.Coxeter.Basic
import ...

variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

Some important functions:

- `cs.simple : B → W`
- `cs.wordProd : List B → W`
- `cs.length : W → ℕ`

Some definitions in mathlib

```
import Mathlib.GroupTheory.Coxeter.Basic
import ...

variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

Some important functions:

- `cs.simple : B → W`
- `cs.wordProd : List B → W`
- `cs.length : W → ℕ`
- `cs.leftInvSeq : List B → List W`

Some theorems in mathlib

```
theorem length_inv (w : W) : cs.length w⁻¹ = cs.length w
```

Some theorems in mathlib

```
theorem length_inv (w : W) : cs.length w⁻¹ = cs.length w

theorem exists_reduced_word (w : W) :
  ∃ (ω : List B), ω.length = cs.length w ∧ w = cs.wordProd ω
```

Some theorems in mathlib

```
theorem length_inv (w : W) : cs.length w⁻¹ = cs.length w

theorem exists_reduced_word (w : W) :
  ∃ (ω : List B), ω.length = cs.length w ∧ w = cs.wordProd ω

theorem getElem_leftInvSeq (ω : List B) (j : ℕ)
  (h : j < ω.length) :
  (cs.leftInvSeq ω)[j]
  = cs.wordProd (List.take j ω) * cs.simple ω[j] * (cs.wordProd
    (List.take j ω))⁻¹
```

Some theorems in mathlib

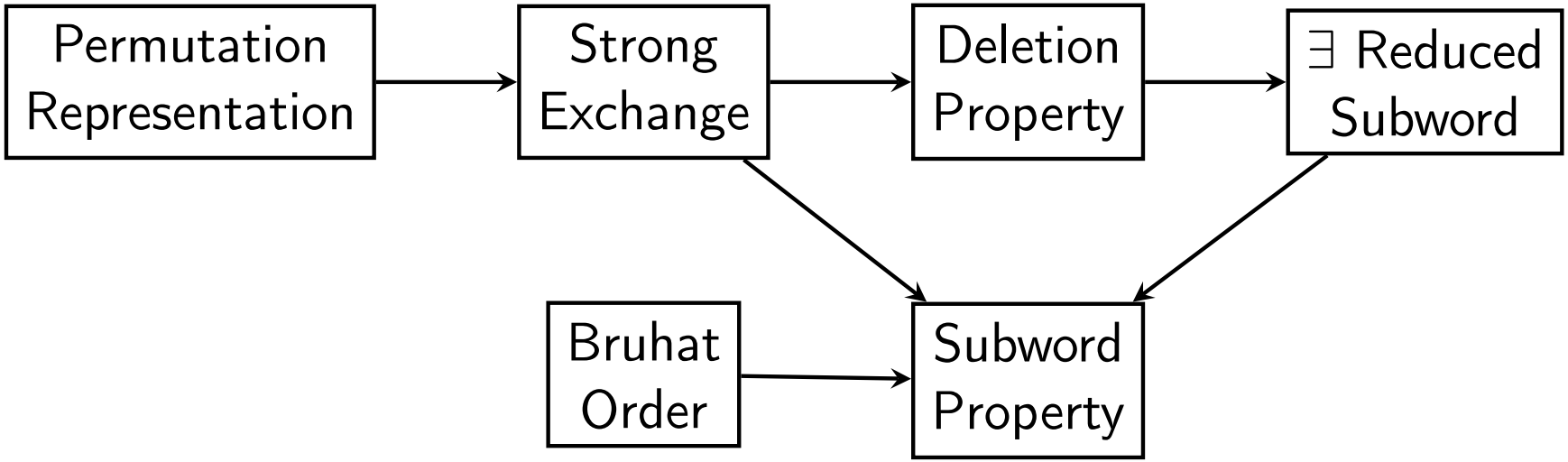
```
theorem length_inv (w : W) : cs.length w⁻¹ = cs.length w

theorem exists_reduced_word (w : W) :
  ∃ (ω : List B), ω.length = cs.length w ∧ w = cs.wordProd ω

theorem getElem_leftInvSeq (ω : List B) (j : ℕ)
  (h : j < ω.length) :
  (cs.leftInvSeq ω)[j]
  = cs.wordProd (List.take j ω) * cs.simple ω[j] * (cs.wordProd
    (List.take j ω))⁻¹

theorem IsReduced.nodup_leftInvSeq {ω : List B}
  (rω : cs.IsReduced ω) : (cs.leftInvSeq ω).Nodup
```

Formalizing more stuff

We'll follow the textbook *Combinatorics of Coxeter Groups* by Björner–Brenti.

Formalizing more stuff

We'll follow the textbook *Combinatorics of Coxeter Groups* by Björner–Brenti.



Packaging things together

Define

```
class CoxeterGroup (W : Type*) extends Group W where
  B : Type*
  M : CoxeterMatrix B
  cs : CoxeterSystem M W
```

Packaging things together

Define

```
class CoxeterGroup (W : Type*) extends Group W where
  B : Type*
  M : CoxeterMatrix B
  cs : CoxeterSystem M W
```

Instead of writing

```
variable {B : Type*} {W : Type*} [Group W] {M : CoxeterMatrix B}
    (cs : CoxeterSystem M W)
```

we can now write instead

```
variable {W : Type*} [CoxeterGroup W]
```

Permutation representation

Define the set of reflections for W (usually denoted T):

```
def ReflectionSet (W : Type*) [CoxeterGroup W] :=
  {t : W // cs.IsReflection t}
```

Permutation representation

Define the set of reflections for W (usually denoted T):

```
def ReflectionSet (W : Type*) [CoxeterGroup W] :=
  {t : W // cs.IsReflection t}
```

and the set of roots ($R = T \times \{\pm 1\}$):

```
def RootSet (W : Type*) [CoxeterGroup W] :=
  ReflectionSet W × ZMod 2
```

Permutation representation

Define the set of reflections for W (usually denoted T):

```
def ReflectionSet (W : Type*) [CoxeterGroup W] :=
  {t : W // cs.IsReflection t}
```

and the set of roots ($R = T \times \{\pm 1\}$):

```
def RootSet (W : Type*) [CoxeterGroup W] :=
  ReflectionSet W × ZMod 2
```

The permutation representation is the action of W on the set of roots R. To define it, we need to first define it on the generating set S, then show that it satisfies the relations.

Permutation representation

Define the set of reflections for W (usually denoted T):

```
def ReflectionSet (W : Type*) [CoxeterGroup W] :=
  {t : W // cs.IsReflection t}
```

and the set of roots ($R = T \times \{\pm 1\}$):

```
def RootSet (W : Type*) [CoxeterGroup W] :=
  ReflectionSet W × ZMod 2
```

The permutation representation is the action of W on the set of roots R. To define it, we need to first define it on the generating set S, then show that it satisfies the relations.
(Omitted.)

Permutation representation

```
def permRep : W →* Equiv.Perm (RootSet W)

def η (w t : W) : ZMod 2 :=
  count t (cs.leftInvSeq (choose (cs.wordProd_surjective w)))

theorem permRep_eq (w : W) (r : RootSet W) :
  permRep w r
  = ⟨⟨MulAut.conj w r.1.val, r.1.prop.conj _⟩, r.2 + η w⁻¹
    r.1.val⟩
```

Permutation representation

```
def permRep : W →* Equiv.Perm (RootSet W)

def η (w t : W) : ZMod 2 :=
  count t (cs.leftInvSeq (choose (cs.wordProd_surjective w)))

theorem permRep_eq (w : W) (r : RootSet W) :
  permRep w r
  = ⟨⟨MulAut.conj w r.1.val, r.1.prop.conj _⟩, r.2 + η w⁻¹
    r.1.val⟩
```

In usual notation:

- $\eta(w, t)$ is the number of occurrences mod 2 of t in $s_1, s_1 s_2 s_1, s_1 s_2 s_3 s_2 s_1, \ldots$ for some choice of word $w = s_1 s_2 \ldots s_k$.
- $\pi_w(t, \varepsilon) = (wtw^{-1}, \varepsilon\eta(w^{-1}, t))$

Permutation representation

η is independent of the choice of word:

```
theorem eta_spec (ω : List (B W)) (t : W) :
  η (cs.wordProd ω) t = count t (cs.leftInvSeq ω)
```

Permutation representation

η is independent of the choice of word:

```
theorem eta_spec (ω : List (B W)) (t : W) :
  η (cs.wordProd ω) t = count t (cs.leftInvSeq ω)
```

The proof of strong exchange uses

```
theorem eta_eq_one_iff {t w : W} :
  η w t = 1 ↔ cs.IsLeftInversion w t
```

Strong exchange & deletion property

```
theorem strong_exchange {ω : List (B W)} {t : W}
  (h : cs.IsLeftInversion (cs.wordProd ω) t) :
  ∃ i < ω.length, t * cs.wordProd ω = cs.wordProd (ω.eraseIdx i)
```

Strong exchange & deletion property

```
theorem strong_exchange {ω : List (B W)} {t : W}
  (h : cs.IsLeftInversion (cs.wordProd ω) t) :
  ∃ i < ω.length, t * cs.wordProd ω = cs.wordProd (ω.eraseIdx i)

theorem deletion_property {ω : List (B W)}
  (hω : ¬ cs.IsReduced ω) :
  ∃ i j, i < j ∧ j < ω.length ∧ cs.wordProd ω = cs.wordProd
    ((ω.eraseIdx j).eraseIdx i)
```

Strong exchange & deletion property

```
theorem strong_exchange {ω : List (B W)} {t : W}
  (h : cs.IsLeftInversion (cs.wordProd ω) t) :
  ∃ i < ω.length, t * cs.wordProd ω = cs.wordProd (ω.eraseIdx i)

theorem deletion_property {ω : List (B W)}
  (hω : ¬ cs.IsReduced ω) :
  ∃ i j, i < j ∧ j < ω.length ∧ cs.wordProd ω = cs.wordProd
    ((ω.eraseIdx j).eraseIdx i)

theorem exists_reduced_subword (ω : List (B W)) :
  ∃ (ω' : List (B W)), ω' <+ ω ∧ cs.IsReduced ω' ∧
    cs.wordProd ω = cs.wordProd ω'
```

Here <+ means 'is a sublist of'.

Defining the Bruhat order

Definition

For $u, w \in W$, $u \leq w$ in Bruhat order means that there is a chain

$$u = u_0 \to u_1 \to \ldots \to u_{k-1} \to u_k = w$$

where for each i, $u_{i+1} u_i^{-1}$ is a reflection and $\ell(u_i) < \ell(u_{i+1})$.

Defining the Bruhat order

Definition

For $u, w \in W$, $u \leq w$ in Bruhat order means that there is a chain

$$u = u_0 \to u_1 \to \ldots \to u_{k-1} \to u_k = w$$

where for each i, $u_{i+1}u_i^{-1}$ is a reflection and $\ell(u_i) < \ell(u_{i+1})$.

```
inductive le : W → W → Prop
  | rfl (u : W) : le u u
  | step (u v w : W) (h1 : le u v)
      (h2 : cs.IsReflection (w * v⁻¹))
      (h3 : cs.length v < cs.length w) : le u w
```

Defining the Bruhat order

Definition

For $u, w \in W$, $u \leq w$ in Bruhat order means that there is a chain

$$u = u_0 \to u_1 \to \ldots \to u_{k-1} \to u_k = w$$

where for each i, $u_{i+1}u_i^{-1}$ is a reflection and $\ell(u_i) < \ell(u_{i+1})$.

```
inductive le : W → W → Prop
  | rfl (u : W) : le u u
  | step (u v w : W) (h1 : le u v)
      (h2 : cs.IsReflection (w * v⁻¹))
      (h3 : cs.length v < cs.length w) : le u w
```

The following lets us write `u ≤ w` instead of `le u w`:

```
instance : PartialOrder W
```

Defining the Bruhat order

Definition

For $u, w \in W$, $u \leq w$ in Bruhat order means that there is a chain

$$u = u_0 \to u_1 \to \ldots \to u_{k-1} \to u_k = w$$

where for each i, $u_{i+1}u_i^{-1}$ is a reflection and $\ell(u_i) < \ell(u_{i+1})$.

```
inductive le : W → W → Prop
  | rfl (u : W) : le u u
  | step (u v w : W) (h1 : le u v)
      (h2 : cs.IsReflection (w * v⁻¹))
      (h3 : cs.length v < cs.length w) : le u w
```

The following lets us write `u ≤ w` instead of `le u w`:

```
instance : PartialOrder W
```

Note: `le` depends on `cs`. This is why it is useful to have `[CoxeterGroup W]`, so that Lean knows which `CoxeterSystem` to use when we write `u ≤ w`.

Subword property

For convenience, we define

```
def ReducedWord (w : W) :=
  {ω : List (B W) // cs.IsReduced ω ∧ w = cs.wordProd ω}
```

Subword property

For convenience, we define

```
def ReducedWord (w : W) :=
  {ω : List (B W) // cs.IsReduced ω ∧ w = cs.wordProd ω}
```

Finally ...

```
theorem subword_property (u : W) {w : W} (ω : ReducedWord w) :
  u ≤ w ↔ ∃ (μ : ReducedWord u), μ.val <+ ω.val
```

Subword property

For convenience, we define

```
def ReducedWord (w : W) :=
  {ω : List (B W) // cs.IsReduced ω ∧ w = cs.wordProd ω}
```

Finally ...

```
theorem subword_property (u : W) {w : W} (ω : ReducedWord w) :
  u ≤ w ↔ ∃ (μ : ReducedWord u), μ.val <+ ω.val
```

This lets us prove that Bruhat intervals are finite:

```
theorem finite_Icc (u w : W) : Finite (Set.Icc u w)

instance : LocallyFiniteOrder W

theorem card_Icc_le (u w : W) :
  Finset.card (Finset.Icc u w) ≤ 2 ^ cs.length w
```

Github page

The page for this project can be found at

https://github.com/mlyean/lean-coxeter

Further directions

Further directions

- Construct the geometric representation

Further directions

- Construct the geometric representation
- Prove that symmetric groups are Coxeter groups

Further directions

- Construct the geometric representation
- Prove that symmetric groups are Coxeter groups
- Prove that Coxeter groups are characterized by the exchange property

Further directions

- Construct the geometric representation
- Prove that symmetric groups are Coxeter groups
- Prove that Coxeter groups are characterized by the exchange property
- Classify finite Coxeter groups

Further directions

- Construct the geometric representation
- Prove that symmetric groups are Coxeter groups
- Prove that Coxeter groups are characterized by the exchange property
- Classify finite Coxeter groups
- ...